

WEEK-7

User stories

A user story is a tool in Agile software development used to capture a description of a software feature from a user's perspective. The user story describes the type of user, what they want and why. A user story helps to create a simplified description of a requirement.

The purpose of a user story is to write down how a project will deliver value back to the end user.

User stories do not replace use cases or technical requirements documentation. Instead, user stories can be written by product developers to help prioritize how functionality is going to be added to a project over the project timeframe. A user story can be considered a starting point to a conversation that establishes the real product requirement.

Characteristics of a user story

A user story template often follows the same format. The three components of a user story are:

1. Who- This is typically a job role, customer or type of user, also known as the user persona.
2. What- This is the goal that the user wants the product to accomplish or implement.
3. Why- This is the reason why the user needs the feature or functionality.

The end result is a sentiment like, “As a <who>, I want <what> so that <why>.” Further detail can be added to a user story by breaking it into smaller user stories and grouping them into themes.

An Agile user story is meant to be short, usually fitting on a sticky note or note card. The user stories should be written by the business in the language of the customer so that it is clear to both the business and the development team what the customer wants and why they want it.

In some cases, user stories are also assigned a unique identifier and an effort/priority level. The unique identifier is typically a number and allows developers to keep track of how many user stories there are and when they are completed. The effort or priority level is more customized to the team but is typically a range that signifies how long the feature will take, how many developers will be needed or how many requirements the feature has.

Lastly, user stories should be associated with pre-defined acceptance criteria. Acceptance criteria are used to identify the boundaries of a user story and what needs to be done in order for the story to be considered complete. This could also include any tests that need to be done to verify a user story.

Examples of user stories

Following the above format, a few examples of a user story are:

- As Max, I want to invite my friends, so we can enjoy this service together.
- As Sascha, I want to organize my work, so I can feel more in control.
- As a manager, I want to be able to understand my colleagues progress, so I can better report our success and failures.
- As a user, I want to upload photos so that I can share photos with others.
- As an administrator, I want to approve photos before they are posted so that I can make sure they are appropriate.
- As a **social media manager**, I want to tag the photos under specific categories so that I can filter and search the photos for future use.

By: Diane Hoffman

A Story 📄	An Epic 📁
The smallest unit of work; can't be split	Can be broken down into more specific and smaller elements - User Stories

Fit a shippable product increment that should be delivered during 1 sprint	May be implemented during a few sprints
Represents some value that user will get after implementation	Indicates a more general task (for example, implementation of a whole user-flow)
Quite easy to estimate	Harder to estimate since the scope is flexible

Epics provide us with a high-level view of our goals and how we're moving towards them. It also helps us during the prioritization process since we can check which Epics require our attention the most and, therefore, which Stories should be implemented first.

As a passenger, I want several available drivers to be displayed so that I can choose the most suitable option for me.

What acceptance criteria can be applied to this Story?

- The app shows drivers that were online within last 20 minutes and don't have an ongoing ride.
- The app shows only 5 drivers that are closest to the user.
- A user can browse profiles of these drivers, including their photos and rates.

Points to Remember while Creating User Stories:

1. Validating user needs

At first, you need to define the users who will use your product/service and create buyer personas. A buyer persona is a semi-fictional character of your ideal user.

You should analyze what problems your buyer personas are facing and define the ways your product will help them to solve their challenges. At this stage, you may

conduct interviews with target users, understand their pain points, ask appropriate questions, etc.

2. Creating epics

Writing epics will help you sketch your product's key functionality without going deep in the details.

Here's an example:

You're creating a dating app for users who can to create, manage and edit their profiles. So, the epic is "Managing user profile".

User stories are:

- *As a user, I want to write my bio info so I can keep it up to date.*
- *As a user, I want to add dating preferences so my contacts will define me easier.*
- *As a user, I want to add the cover picture to my page so my profile will look bright and attractive.*

Epics assist to track large ideas in a product backlog without the need to overpopulate it with multiple items. You may use Burndown charts to visualize epics as they keep all stakeholders informed and make the development team motivated.

3. Creating user stories according to the user types

Your next step is diving into the details to break the epics down into the stories.

4. Choosing a tool for visualizing user stories

Ideally, all user stories should be visible for each stakeholder of the project. That's why you need an appropriate multifunctional project management tool.

User stories mapping can be visualized with convenient boards and cards.

5. Defining acceptance criteria for stories

Acceptance criteria outline the boundaries of user stories. They are used to confirm when a story is completed and working as intended.

These criteria can be functional (identifying user tasks, functions or business processes), non-functional (design and UI-related) and performance criteria (related to performance metrics, for example, loading time).

Three C's of User Stories?

Before moving ahead with understanding the life cycle of a user story in Agile development methodology, here are the three C's of user stories that help define them better.

1. Cards

These are the post-its or any small story cards that briefly explain the intent of the user story. Keeping the limited space of the cards in mind, it would be a good practice to stick to the user story name, rather than defining it.

An example of such a description could be – “Paypal Payment Integration”. This card description is short and crisp and also conveys the intent of the user story perfectly.

2. Conversations

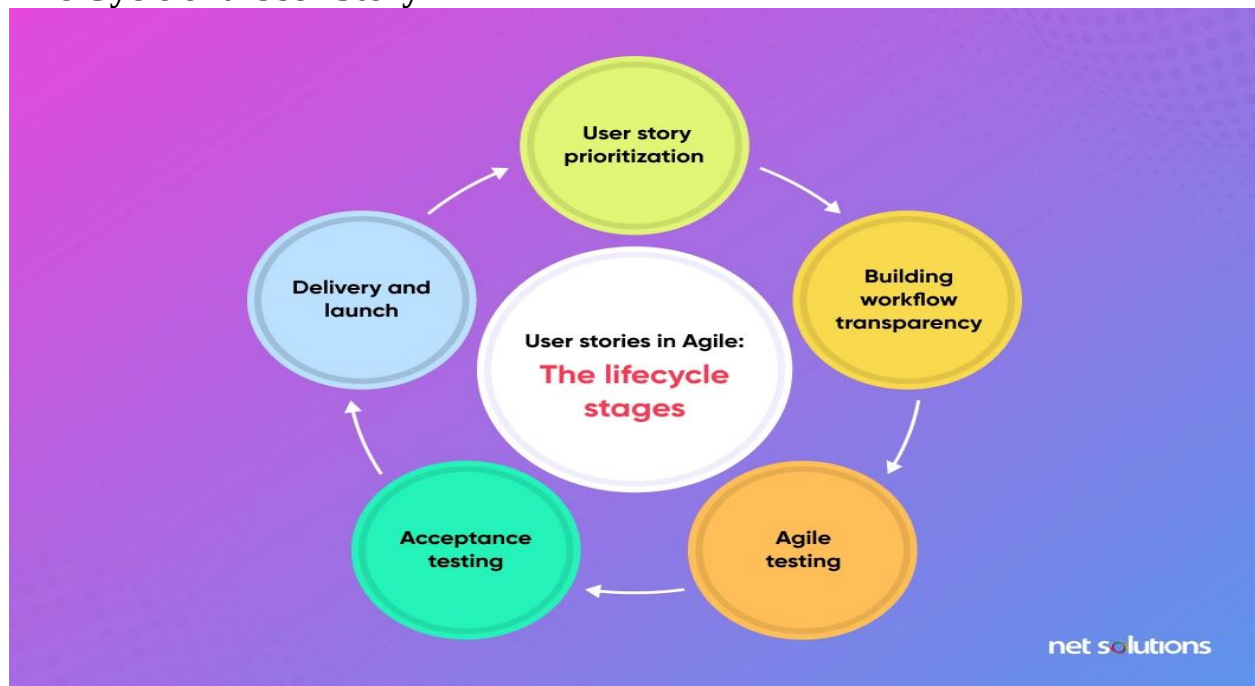
The product development team comprises designers, developers, business analysts, product managers, product owners, quality analysts, and testers.

Communication within this cross-functional setup gives clarity of user pain points, the priority of solving them, the solution, the expectations from team members, etc. The teams arrange daily stand-ups to discuss user stories over the sprint life cycle and have productive conversations towards delivering speed and quality.

3. Confirmation

The objective of the confirmation phase is to roll down the acceptance criteria for the shortlisted user stories. This would ensure that every user story has some quality parameters to adhere to, and they are implemented successfully.

Life Cycle of a User Story



1. User story prioritization:

Ranking user stories based on the urgency of solving the problem

Once the product owner has assembled the different types of user stories on the cards or post-its, the next task is to prioritize them. There might be tens or hundreds of stories, and the Agile development team can't be acting on all of them at once; hence it's important to align them based on the urgency.

The popular method of user story prioritization is:

MoSCoW Prioritization:

MoSCoW stands for — Must-haves, should-haves, could-haves, and will-not haves in terms of features. User stories in Agile circle around this prioritization technique to ensure that the customer's needs are taken care of with respect to urgency. Let's discuss each of the MoSCoW parameters in detail:

- **Must-Have Features:** These features should be included (at any cost) in the software on a priority basis. These features define the product and form its foundation.
- **Should-Have Features:** These are the features that need to be added to the software and add significant value. However, these can be prioritized once the must-haves have been developed and delivered.

- **Could-Have Features:** These are the features that can improve the product but can be avoided if the budget or the delivery timelines are restrictive. However, the development team can work on them after the building and launching MVP.
- **Will-Not-Have Features:** These features offer less value and can be omitted from the product backlog. In general, these are the features that customers care less about.

For example, social login integrations may demand higher priority (a must-have) than a PayPal payment integration (a should-have) for a particular product. It is essential to sit along with the team and hold discussions over prioritizing the tasks at this initial stage.

2. Building Workflow Transparency

Goal: Building awareness around the progress of the user stories

The sprint backlog is already in place for all the user stories in the Agile software development process, in order of their preference. How does the product owner or the product manager know which user stories are being worked upon?

This problem is solved by setting the status for each of the user stories. The development team can manage the status of the user story on a digital sprint task board. It will help maintain transparency by visualizing the flow and the progress of user stories.

Moreover, it can also help estimate the gap between completed and pending user stories, which further helps maintain the product backlog.

3. Agile Testing

Goal: To check the efficiency of the completed user story

The Agile development company might have done their bit with implementing the user stories. But do they work? Do they offer the solution that end-users expect? The answers to these and similar such questions lie with the Agile testing team that checks the feature's workability and efficiency.

The testing team runs all sorts of quality tests such as unit testing, integration testing (with the flow of other user stories incorporated), functional testing, acceptance tests, etc.

In case of any bugs, the user story development process is iterated, to incorporate the changes. The team can again pick the tasks from the sprint backlog while the other user stories are put on hold.

4. Acceptance Testing

Goal: Ensuring the user story adheres to the INVEST Standards and user acceptance testing

To ensure the delivery of a good story, the team needs to ensure that it adheres to the INVEST standards, which stands for:

- **Independent:** The user stories should be independent of other user stories. The “As I, I want, and so that” of a particular user story should not depend on another user story.
- **Negotiable:** The user stories should not be considered final at any stage of the product development process as the definition of a feature can change at any time. Thus, the user stories should be left open for review.
- **Valuable:** The prioritized user stories should add value to the end-user, i.e., the prioritized user stories should add value to the end-user, i.e., the user problems need to be solved effectively.
- **Estimable:** The size and the scope of the user story should be measurable. This will help in sub-categorizing the large user stories (epics) into smaller and manageable branched out tasks.
- **Small:** The user story size should be small, i.e., the team should develop it in one or two sprint cycles.
- **Testable:** It should be possible to test and deploy individual user stories without interference.

Another way to validate completed user stories is to run user acceptance tests with your end-users.

The format for acceptance criteria – “Given[a situation], When[describes an action], Then[user expectation].”

Example of Agile User Story Acceptance Criteria Format		
Given	A workable cloud-based CRM system	Intent: Overview of a situation (some context)
When	I communicate with ten leads in a day	Intent: Explains some action that the user intends to carry out
Then	I should be able to record them in the CRM in real-time	Intent: Mentions the consequence of the action (which should satisfy the user goals)

Thus, we can write the complete user story from initialization to acceptance in the following format:

- As a: Salesperson
- I want: a customer relationship management software
- So that: manage all my client conversations in one place
- Given: A workable cloud-based CRM system
- When: I communicate with ten leads in a day
- Then: I should be able to record them in the CRM in real-time

Now that is a complete and connected user story that the Agile Development team needs to work on. If the story looks disconnected at any point, the user story needs to be reframed again over the sprint cycles until everything makes sense.

5. Delivery and Launch

Goal– To deliver a user story and move on to the next one

At this stage, the agile lifecycle of a user story ends. This means that the project manager has approved, the quality assurance team has given the go-ahead, and user stories in agile adhere to the INVEST standards.

When the must-have user stories are implemented and tested, it's the ideal time to consider launching the MVP (Minimum Viable Product).

Post the launch — the Agile team can move over to the next user stories in the queue, i.e., with the should-haves and could-haves and the other incoming user stories based on customer feedback.

Benefits of user stories

User stories provide development teams with important context before a project even begins. They place emphasis on the user and focus on solving real situations a customer might face. This can help development teams think more critically and creatively. Additional benefits of using user stories include:

- Increased visibility and collaboration across the development team.
- Better use of end user or customer feedback.
- Can save time when prioritizing the development of requirements and functionality.
- Helps avoid restrictions that occur when specification details are defined too early on.
- Higher clarity around business value and delivering products that end users actually need.

Benefits of User Stories in Agile

The benefits of framing user stories in agile include:

- Facilitates a better understanding of the features that make up the product

- The defined feature set of the product is customer-centric, which means that the requirements are enlisted from the user's perspective
- User stories boost productivity. When a story is complete, the team feels a sense of accomplishment and enthusiastically moves towards completing the user stories in the backlog.
- Enables innovative Ideas! When the team has a better understanding of what they need to accomplish, they can think around creative ways to achieve it, which, in turn, helps disrupt the market.